

Discrete Structure

Unit 3. Automata Theory

3.1 Finite State Automata:

A **Finite Automata** (FA), also known as a **Finite State Machine (FSM)**, is an abstract mathematical model used to recognize patterns or formal languages. It is the simplest model of computation, consisting of a finite set of states and rules for transitioning between them based on input symbols.

A finite automaton can be defined as a 5-tuple:

$M = \{ Q, \Sigma, q, F, \delta \}$, where:

- Q : Finite set of states
- Σ : Set of input symbols
- q : Initial state
- F : Set of final states
- δ : Transition function can be express $\delta: Q \times \Sigma \rightarrow Q$

Types of Finite Automata

There are two types of finite automata:

- Deterministic Finite Automata (DFA)
- Non-Deterministic Finite Automata (NFA)

1. Deterministic Finite Automata (DFA)

In a DFA, **for each state and each input symbol, there is exactly one transition** to a next state. **Null (ϵ) transitions are not allowed**, and a transition must be defined for **every input symbol in every state**.

DFA consists of 5 tuples $\{Q, \Sigma, q, F, \delta\}$.

Q : set of all states.

Σ : set of input symbols. (Symbols which machine takes as input)

q : Initial state. (Starting state of a machine)

F : set of final state.

δ : Transition Function, defined as $\delta : Q \times \Sigma \rightarrow Q$.

Example:

Construct a DFA that accepts all strings ending with 'a'.

Given:

$\Sigma = \{a, b\}$,

$Q = \{q_0, q_1\}$,

$F = \{q_1\}$

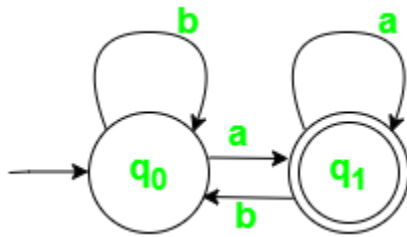


Fig 1. State Transition Diagram for DFA with $\Sigma = \{a, b\}$

State \ Symbol	a	b
q ₀	q ₁	q ₀
q ₁	q ₁	q ₀

In this example, if the string ends in 'a', the machine reaches state q₁, which is an accepting state.

2) Non-Deterministic Finite Automaton (NFA),

In a **Non-Deterministic Finite Automaton (NFA)**, for a given input symbol, the machine can move to **one or more states**, or even **no state**. Therefore, the next state is **not uniquely determined**, which is why it is called **non-deterministic**.

Formal Definition

An NFA is represented by a 5-tuple

$$(Q, \Sigma, \delta, q_0, F)$$

where:

- Q = finite set of states
- Σ = finite input alphabet

- $\delta: Q \times (\Sigma \cup \{\epsilon\}) \rightarrow 2^Q$
(Transitions may include **ϵ -moves**, and may lead to multiple states)
- $q_0 \in Q$ = initial state
- $F \subseteq Q$ = set of final states
- Construct an NFA that accepts strings ending in 'a'.
- Given:
- $\Sigma = \{a, b\}$,
- $Q = \{q_0, q_1\}$,
- $F = \{q_1\}$

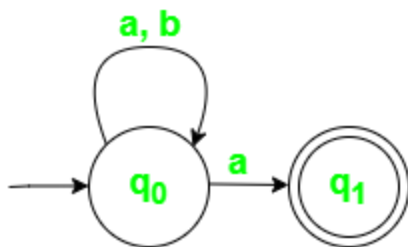


Fig 2. State Transition Diagram for NFA with $\Sigma = \{a, b\}$

- State Transition Table for above Automaton,

State \ Symbol	a	b
q_0	$\{q_0, q_1\}$	q_0
q_1	$\varnothing$	$\varnothing$

- In an NFA, if any transition leads to an accepting state, the string is accepted.

- **NFA vs DFA**

DFA	NFA
Deterministic: exactly one transition per input symbol	Non-deterministic: multiple transitions per input symbol allowed
No ϵ (null) moves	Allows ϵ (null) moves
Simpler but sometimes larger in design	More flexible and easier to design
Next state is uniquely determined	Next state may be multiple possibilities
Recognizes regular languages; NFAs can be converted to DFA	Recognizes regular languages; equivalent in power to DFA